

Delivery

Handoff / maintenance doc · current as of 2026-04-22

WHAT IT DOES

Brief: Daily produce check-in web app for the warehouse team.

General: An admin uploads the daily delivery worksheet (PDF from IT) to Firebase Storage. A Python backend parses it with `pdfplumber`, extracts supplier blocks and line items (including floor-pull quantities flagged with an asterisk), and writes a structured record to Firestore. Warehouse staff open the app on phones/iPads, walk the supplier sections, and mark each item as received/short/over/return, optionally confirming pull quantities. A separate daily CSV carries basement/floor location metadata, and a second PDF carries a 3-day high-count forecast — both are parsed and joined to the worksheet. At completion, an exception report listing discrepancies + pulls is snapshotted for the buyer. (An IT-driven consolidation to a single combined CSV is in planning — see `NEW_CSV_FORMAT.md`.)

TECH STACK

Languages	Python 3.9+ (backend), vanilla JavaScript / HTML / CSS (frontend — no build step)
Framework	FastAPI + uvicorn; Pydantic models; Dockerfile-based container
Libraries	pdfplumber (PDF text extraction), PyMuPDF, firebase-admin (Firestore + Storage), python-multipart
Backend data	Google Firestore (deliveries, items, daily logs, exception reports); Firebase Storage (uploaded PDFs/CSVs)
Frontend	Single-page app served as static assets from FastAPI; cache-buster <code>?v=__COMMIT__</code> replaced server-side with the git hash at serve time

HOSTING & REMOTE RESOURCES

Live site	<code>https://delivery-app-481756503401.us-east1.run.app</code>
Cloud Run	project <code>delivery-worksheet-app</code> · region <code>us-east1</code> · service <code>delivery-app</code>
Firebase project	<code>delivery-worksheet-app</code>
Storage bucket	<code>delivery-worksheet-app.firebaseio.com</code>
Firestore	(default database, same project)
Secret Manager	secret <code>firebase-service-account</code> — Firebase admin credentials, mounted into Cloud Run at runtime
GitHub repo	<code>https://github.com/kjmacgrub/Delivery</code>
Local checkout	<code>/Users/km/python3/source/Delivery</code>

REPO LAYOUT — FILES THAT MATTER

<code>delivery/api/app.py</code>	FastAPI app factory; registers routers; serves the SPA static files.
<code>delivery/api/routes/</code>	HTTP handlers: <code>deliveries.py</code> , <code>items.py</code> , <code>checkin.py</code> , <code>notes.py</code> , <code>storage.py</code> , <code>daily_logs.py</code> .
<code>delivery/parser/</code>	PDF/CSV parsers: <code>pdf_parser.py</code> (main worksheet), <code>high_count_parser.py</code> , <code>inventory_parser.py</code> , <code>csv_parser.py</code> , <code>product_parser.py</code> (description → structured fields).
<code>delivery/services/</code>	Business logic + persistence: <code>delivery_service.py</code> (Firestore ops), <code>firebase_storage.py</code> , <code>daily_log_service.py</code> .
<code>delivery/models.py</code>	Pydantic data models (Delivery, SupplierEntry, LineItem, statuses, reports).
<code>delivery/schemas.py</code>	Shared schema definitions.
<code>delivery/firebase_client.py</code>	Firebase admin SDK init.
<code>delivery/static/</code>	Frontend: <code>index.html</code> , <code>app.js</code> , <code>styles.css</code> . No build step — edits picked up on hard refresh.
<code>Dockerfile</code>	Container image built by Cloud Build during <code>./deploy.sh</code> .
<code>deploy.sh</code>	One-shot deploy script (also has a <code>setup</code> subcommand for first-time project wiring).

<code>pyproject.toml</code>	Python dependencies + packaging.
<code>firestore.rules</code> / <code>firebase.json</code>	Firestore security rules and Firebase tooling config.
<code>firebase-service-account.json</code>	Service-account credentials for local runs (git-ignored in principle; not committed for prod use — Secret Manager holds prod copy).
<code>NEW_CSV_FORMAT.md</code>	In-progress transition plan for consolidated daily CSV (supersedes 4 current files).

FIREBASE / FIRESTORE DATA SHAPE

Storage paths	<code>delivery-files/incoming/</code> (uploaded PDFs/CSVs awaiting parse) · <code>delivery-files/processed/</code> · <code>delivery-files/reports/</code> (exception snapshots)
Firestore collections	<code>deliveries</code> (one doc per delivery day, contains supplier entries and line items) · daily logs and exception reports as subcollections / separate collections

DEVELOP & DEPLOY

Setup (once)	<code>python3 -m venv .venv && .venv/bin/pip install -e .</code>
Run locally	<code>.venv/bin/python -m uvicorn delivery.api.app:create_app --factory --host 0.0.0.0 --port 8001 --reload</code>
Local URL	<code>http://localhost:8001</code> (8000 is reserved for a different app)
Tests	<code>.venv/bin/pytest</code> — tests in <code>tests/</code> (optional dev dep)
Deploy	<code>./deploy.sh</code> — builds Docker image via Cloud Build, deploys new Cloud Run revision, routes 100% traffic.
Version display	UI shows <code>v1.XX</code> · <code><commit-hash></code> from <code>VERSION_HISTORY</code> in <code>app.js</code> ; cache-busters auto-injected.

ACCOUNTS REQUIRED TO MAINTAIN

- **GitHub** (kjmactgrub/Delivery) — collaborator access for code.
- **Google Cloud** (project `delivery-worksheet-app`) — Cloud Run, Cloud Build, Secret Manager, IAM.
- **Firebase console** (same project) — Firestore, Storage, security rules.
- No CI pipeline; no frontend build step; pytest is optional and run manually.